---

# □ 2. Methods & Parameter Passing (DSA CORE)

## A. Method Definition & Call

### Problem 2.1 – Call Order Awareness

**Task:**
Write three methods:

- `start()`
- `process()`
- `end()`

`start()` should call `process()`, and `process()` should call `end()`.

**Goal:**
Print messages so that the output proves you understand **call flow**.

□ *DSA relevance:* Helper methods call each other recursively or sequentially.

---

### Problem 2.2 – Missing Call Bug

**Task:**
Write a method `calculate()` that prints `"Calculating..."`.

Write `main()` but **forget to call** `calculate()`.

**Question:**
Why does nothing print?

□ *DSA relevance:* Many bugs are due to forgetting method calls.

---

## B. Return Values (VERY IMPORTANT)

### Problem 2.3 – Return vs Print

**Task:**
Write a method `sum(int a, int b)`:

- Version 1: prints sum
- Version 2: returns sum

Use both from `main()`.

**Question:**
Which version is usable inside another calculation and why?

 *DSA relevance:* DSA algorithms **depend on returned values**, not printing.

---

### Problem 2.4 – Broken Return Path

**Task:**
Write a method `isEven(int n)` returning `boolean`.

✗Intentionally forget to return in one condition.

**Question:**
Why does Java refuse to compile?

 *DSA relevance:* All recursive and decision-based methods must return correctly.

---

# C. Pass-by-Value (THIS CONFUSES EVERYONE)

### Problem 2.5 – Primitive Trap

**Task:**

```
void change(int x) {
    x = 100;
}
```

Call this method from `main()` with `int a = 10`.

**Predict Output:**
What prints: `a`?

 *You must explain WHY.*

---

### Problem 2.6 – Swap Failure

**Task:**
Try to swap two integers using a method.

**Question:**
Why does swapping fail?

 *DSA relevance:* Understanding why primitive swaps fail explains why arrays are used.

---

# D. Method Overloading

### Problem 2.7 – Overload Resolution

**Task:**
Create overloaded methods:

- `print(int x)`
- `print(double x)`
- `print(int x, int y)`

Call them with different arguments.

**Question:**
How does Java decide which one to call?

---

### Problem 2.8 – Ambiguous Call

**Task:**
Create:

```
print(float x)
print(double x)
```

Call `print(10)`.

**Question:**
Why does Java choose one over the other?

 *DSA relevance:* Wrong overload = wrong logic execution.

---

# 🔥 3. ARRAYS (EXTREMELY IMPORTANT – DSA BEGINS HERE)

## A. 1D Arrays

### Problem 3.1 – Manual Traversal

**Task:**
Given an array, print:

- index
- value
- index + value

🧠 *DSA relevance:* Index-based thinking.

---

### Problem 3.2 – Reverse Print

**Task:**
Print array elements in reverse **without creating a new array**.

🧠 *In-place logic starts here.*

---

### Problem 3.3 – Count Problems

**Task:**
Count:

- even numbers
- odd numbers
- zeros

🧠 *DSA relevance:* Frequency-based problems.

---

## B. In-Place Modification (CRITICAL)

### Problem 3.4 – Modify Original Array

**Task:**
Multiply every element by 2 **inside the same array**.

✖No new array allowed.

 *DSA relevance:* Space optimization.

---

### Problem 3.5 – Replace Rules

**Task:**
Replace:

- even → 0
- odd → 1

 *DSA relevance:* Condition-based transformations.

---

## C. Passing Arrays to Methods

### Problem 3.6 – Reference Reality

**Task:**
Pass an array to a method and change its first element.

**Question:**
Why does the change reflect in `main()` but primitive change didn't?

 If you can't answer this clearly, **you are not DSA-ready**.

---

### Problem 3.7 – Partial Modification

**Task:**
Modify only elements at even indices using a helper method.

 *DSA relevance:* Clean modular code.

---

# D. 2D Arrays (Matrix Thinking)

### Problem 3.8 – Row-wise Traversal

**Task:**
Print a 2D array row by row.

 *Foundation of matrix problems.*

---

### Problem 3.9 – Column-wise Traversal

**Task:**
Print column by column.

 *This breaks beginners.*

---

### Problem 3.10 – Diagonal Sum

**Task:**
Find sum of:

- main diagonal
- secondary diagonal

 *Appears frequently in DSA tests.*

---

# Reality Check (Important)

If you:

- Can’t predict output before running
- Confuse print vs return
- Don’t understand why arrays behave differently than primitives

 **You are NOT wrong**, but
 **You must stop and fix this before DSA**.

---

# Next Step (DO NOT SKIP)

If you want, next I will give:
✅**Exact expected outputs**
✅**Trick questions that fail**